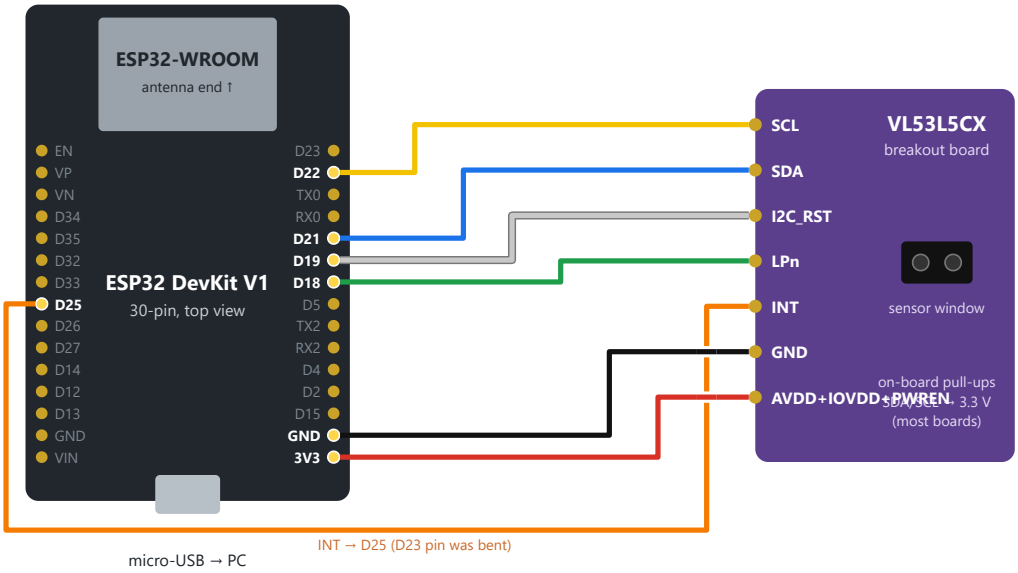
VL53L5CX ↔ ESP32: hand-distance test

Practice build · classic ESP32 DevKit V1 (30-pin) · VL53L5CX 8×8 time-of-flight "lidar" sensor · powered from the PC over USB

> **Before you start:** unplug the robot's battery and pull the motor-driver wires off the ESP32. For this test the ESP32 is powered **only by the USB cable**. Peel the protective film off the sensor window.



Wire list (7 wires)

#	Suggested colour	ESP32 pin	Sensor pin (name on your board may differ)	What it does
1	red	**3V3**	**VIN** / 3V3, or on 9-pin boards **AVDD + IOVDD + PWREN, all three**	Power, 3.3 V. **AVDD is mandatory** (laser + measuring core); IOVDD sets the logic level. Never 5 V on a 3.3 V-only board
2	black	**GND**	**GND**	Common reference (0 V)
3	blue	**D21**	**SDA**	I^2C data, both directions
4	yellow	**D22**	**SCL**	I^2C clock, driven by the ESP32
5	green	**D18**	**LPn**	Must be HIGH, or the sensor ignores I^2C (the sketch sets it HIGH)
6	white	**D19**	**I2C_RST** / RST	Reset, active HIGH, so it must stay LOW (the sketch sets it LOW)
7	orange	**D25** (D23 was bent)	**INT** / GPIO1	Sensor pulls it LOW when a new frame is ready, so the ESP32 gets an interrupt
–	part of wire 1 (3V3)		**PWREN** / PWR_EN	Switches the board's own regulator on (details on page 3)

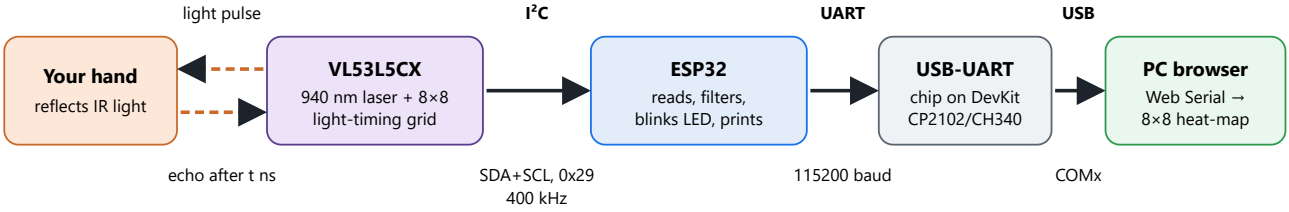
Each of the four control pins (LPn, I2C_RST, INT, PWREN) is explained in detail on page 3. If your breakout lacks one, just skip that wire. Using the black expansion shield? Use the **S** pin of 21/22/18/19, and take power from a **3V3** pin. If you use the **V** pins of the G-V-S rows, set **JUMP to 3.3V** first and measure V–G = 3.3 V before you connect the sensor.

Check before plugging in USB

☐ Multimeter, continuity mode: ESP32 3V3 → sensor VIN beeps; ESP32 GND → sensor GND beeps; 3V3 → GND does **NOT** beep
☐ SDA → 21 and SCL → 22, not swapped. Swapped is the most common mistake, and the scan then finds nothing
☐ Robot battery disconnected; nothing else on 3V3/GND
☐ After plugging in: the ESP32's red power LED is on and the sensor's LED (if it has one) is on. Touch the sensor: it must not get warm

How it works: from your hand to the screen

Every block below is something you just wired, or software that runs on it.



1 · Power

The USB cable gives **5 V**. The regulator on the DevKit (AMS1117) turns it into **3.3 V** for the ESP32 and, through the red wire, for the sensor. The sensor draws about **100 mA** while measuring; the regulator can supply ~800 mA, so there's plenty of headroom. Black GND is the shared 0 V that all signals are measured against. Without it, no communication works.

2 · Measuring: time of flight

A tiny infrared laser (VCSEL, 940 nm, invisible, Class 1 eye-safe) flashes light across a **45° × 45°** cone. A receiver grid of single-photon detectors (SPADs) is split into **8 × 8 = 64 zones**. Each zone times how long the light took to come back:

$$\textbf{distance} = c \cdot t / 2$$

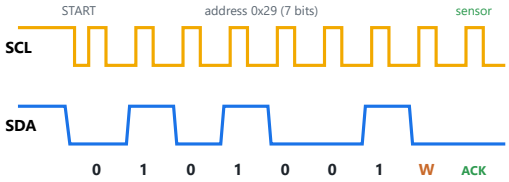
c = 300 000 km/s, and /2 because the light goes there *and* back. A hand at **10 cm** → t ≈ **0.67 ns**. 1 m → 6.7 ns. The sensor builds a histogram of millions of photon arrivals, so it handles this timing itself. Each zone also gets a **status**: 5 or 9 = valid, anything else = ignore.

8×8 mode: **15 frames per second** (one every ~66 ms). Range ≈ 2 cm to 4 m.

3 · Talking: the I²C bus

Two wires, shared. **SCL** = clock, always driven by the ESP32 (the "controller"). **SDA** = data, driven by whoever is talking. Both lines are **open-drain**: chips can only *pull them down to 0 V*. The **pull-up resistors** on the sensor board pull them back up to 3.3 V. That's why it's safe to share the bus and why the pull-ups must go to 3.3 V, not 5 V.

Every message starts with the sensor's address. The ESP32 says "**0x29**, I want to write", and the sensor answers with an ACK by pulling SDA low on the 9th clock:



START = SDA falls while SCL is high. A bit is read while SCL is high. After the address come a 16-bit register number and the data bytes, then STOP.

4 · What happens when you plug it in (the sketch, step by step)

Step	ESP32 does	Why / what you see
1	LPn HIGH · I2C_RST pulse · INT armed	Enables the sensor's I²C, clears any half-finished transfer, and arms the "frame ready" interrupt
2	`Wire.begin(21, 22)` at 400 kHz, I²C scan	Serial prints `0x29`. If nothing shows: wiring, power or LPn
3	`tof.begin()`: uploads **~84 KB firmware** into the sensor	The sensor keeps no firmware between power cycles, so this happens on every boot. Takes a few seconds
4	Set 8×8 resolution, 15 Hz, start ranging	The laser starts flashing (a phone camera may show a faint glow)
5	INT falls (every ~66 ms) → read the result block (~1.4 KB, ≈30 ms at 400 kHz)	64 distances in mm + 64 status codes per frame. The web page shows the INT pulse rate (≈15/s)
6	Keep valid zones, find the closest one, blink the blue LED (D2)	Closer hand = faster blink, like a parking sensor
7	Print `F,d0,d1,…,d63` (−1 = no target) over USB	The web page reads this line and paints the 8×8 heat-map

Orientation test: stand behind the sensor (looking where it looks) and hold your hand on your **left**. The left side of the heat-map should light up. If it's mirrored, change `MIRROR_X` in the sketch; up/down → `FLIP_Y`.

The four control pins: LPn · I2C_RST · INT · PWREN

Power (VIN, GND) and data (SDA, SCL) are the obvious wires. These four carry no data. They decide **whether** the sensor listens, **reset** its I²C logic, **tell** the ESP32 that something happened, and **switch** its power.

> **Reading pin names:** a trailing **n** (LP**n**), a bar over the name or a "#" means **active-LOW**: the function is ON when the pin is at 0 V. No "n" (I2C_RST, PWREN) means **active-HIGH**: ON at 3.3 V. "HIGH" for this sensor means above ≈ 0.7 × IOVDD (≈ 2.3 V); "LOW" means below ≈ 0.3 × IOVDD (≈ 1.0 V). Anything in between is undefined, which is why a **floating** (unconnected, no resistor) input is dangerous: it drifts through that zone with every bit of noise.

Pin	Direction	Active level → effect	State in our build	ESP32	If left floating
LPn	ESP32 → sensor (input)	LOW → I²C interface **off**	HIGH all the time	D18, output	Random "sensor not found" or dropouts
I2C_RST	ESP32 → sensor (input)	HIGH → I²C interface **reset**	10 ms HIGH pulse at boot, then LOW	D19, output	Noise can reset the bus mid-transfer and corrupt frames
INT	sensor → ESP32 (open-drain output)	LOW → "**new frame ready**"	HIGH (pull-up), short LOW pulse every ~66 ms	D25, input + pull-up + interrupt	No pull-up, so it never returns HIGH and is useless
PWREN	3V3 / ESP32 → board regulator	HIGH → sensor **powered**	tied to 3V3 (always on)	– (could be a GPIO)	Board dependent: sensor dead or flickering

LPn: "Low-Power, active-low" = comms enable

What it is: an on/off switch for the sensor's I²C interface. ST calls it *comms enable*. **HIGH**: the chip listens to SDA/SCL. **LOW**: the chip ignores the bus completely, as if it weren't there. It stays **powered**, so its loaded firmware, its settings and its I²C address all survive. Bringing LPn back HIGH does **not** require the 84 KB upload again.

Electrically: a CMOS input referenced to IOVDD (3.3 V). It draws almost no current (µA), so an ESP32 GPIO drives it easily. Good breakouts add a pull-up resistor (often 47 kΩ) so it is HIGH by default; cheap ones sometimes don't. That's why we drive it from **D18** instead of trusting the board.

Why it exists (the real reason): every VL53L5CX powers up at the **same address, 0x29**, and the address is **not stored permanently**. To put 2+ sensors on one I²C bus (e.g. front-left and front-right on the robot), give each sensor its own LPn wire:

1. Hold every LPn LOW, so nobody answers.
2. Raise sensor 1's LPn and tell it "your address is now 0x30".
3. Raise sensor 2's LPn: it's still 0x29, so tell it "0x31". Repeat.
4. Now all LPn are HIGH and each sensor answers at its own address.

In our sketch: `pinMode(18, OUTPUT); digitalWrite(18, HIGH);` runs *before* `Wire.begin` and stays HIGH.

Try it: with the page open, touch a jumper from D18's wire to GND (or change the code). The frames stop, because the ESP32's reads get no ACK. Release it and they come back.

I2C_RST: I²C interface reset

What it is: an **active-HIGH** reset for the sensor's I²C slave logic only, not the measuring core. Drive it 0 → 1 → 0 and the interface returns to "idle, waiting for START".

INT: interrupt, "I have news"

What it is: the only control pin that is an **output** of the sensor. When a frame is finished (every ~66 ms at 15 Hz), the sensor pulls INT **LOW** for a short pulse.

Open-drain: inside the chip there is just a transistor between INT and GND. It can pull the line **down** but never drive it up. Something else must pull it up to HIGH: a resistor on the board, or the ESP32's internal ~45 kΩ pull-up, which we switch on with `INPUT_PULLUP`. Benefits: the HIGH level is whatever the pull-up goes to (3.3 V, perfect for the ESP32), and several sensors can share one INT wire ("wired-OR": any of them can pull it low).

Why use it instead of asking: without INT the ESP32 must **poll**, i.e. keep asking "ready yet?" over I²C. Every question is a bus transaction, wasted time and wasted current. With INT the hardware tells the ESP32 the instant data exists: lower latency, a quiet bus, and the CPU can even **sleep** and be woken by INT (essential for battery devices).

On the ESP32: `attachInterrupt(25, onSensorInt, FALLING)`. On every HIGH→LOW edge the CPU pauses whatever it's doing and runs the tiny **ISR** (interrupt service routine). It only adds 1 to a counter and sets a flag, in microseconds. **Never do I²C inside an ISR** (too slow, and not allowed). `loop()` sees the flag and reads the frame. If INT isn't working, a fallback poll every 20 ms keeps the demo alive.

Check it: the web page shows **INT pulses/s**. ≈ 15 means perfect. 0 with a heat-map still updating means the INT wire or pin is wrong and you are running on the polling fallback.

Why D25: it isn't a boot-strapping pin (0, 2, 5, 12, 15 are), it supports interrupts, and it has an internal pull-up (34–39 don't). *Robot note:* the robot plan used D25 for motor PWMA; that moves to another pin later.

Advanced: ST's driver has *detection thresholds*. The sensor only pulls INT when something is closer than X mm in chosen

Why you need it, the "stuck bus": I²C has no timeouts. Suppose the ESP32 resets (you press EN, upload new code, or a motor causes a brown-out) while the sensor is sending a byte with SDA held LOW. The ESP32 reboots, but the sensor is still powered and still waiting for the clocks that would let it finish. SDA stays LOW forever. A new START needs SDA HIGH, so **the bus is dead** and the scan finds nothing until you unplug USB. A pulse on I2C_RST clears it instantly.

In our sketch: every boot does `HIGH, 10 ms, LOW, 10 ms` on **D19**, then holds it LOW. If the pin floated, noise could fire a reset mid-frame (garbage data or vanishing sensor). Boards usually have a pull-down; with the wire connected we don't rely on it.

zones, so a robot or fall detector can sleep until something actually happens.

PWREN: power enable (board pin)

What it is: not a pin of the chip itself. It exists on some breakouts (generic 9-pin modules, ST's SATEL board) and is the **EN input of the on-board regulator / load switch** that feeds the chip's AVDD and IOVDD. **HIGH** = powered; **LOW** = completely off (µA).

PWREN vs LPn: LPn LOW is "stop listening, stay alive". PWREN LOW is "power gone": firmware, settings and address are lost, and the next start needs the full 84 KB upload (a few seconds) again.

Use: a hard reset if the sensor ever locks up, or zero-power standby in battery projects. **Our build:** tie it to 3V3 if your board has it. If you drive it from a GPIO instead, set it HIGH and wait ≥ 10 ms for the regulator before touching LPn or I²C.

AVDD vs IOVDD (on boards that expose both): AVDD = 3.3 V supply for the analog part and the laser driver. IOVDD = sets the logic level of SDA, SCL, LPn, I2C_RST and INT (1.8 V or 3.3 V). It **must be 3.3 V** to talk to an ESP32. Boards with a single VIN make both on-board.

Power-up timeline (not to scale)

